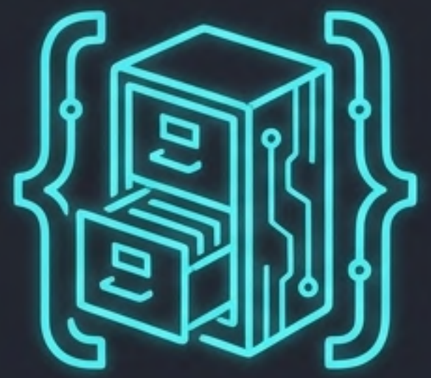
index.js

Maestría en JavaScript: Objects, Math y Strings

Guía de estudio progresiva. De estructuras de datos a utilidades globales y **manipulación de primitivos.**

```
const masterJS = {
    objects: 'Estructuras complejas',
    math: 'El cerebro matemático',
    strings: 'Manipulación de texto',
    ready: true
};
```

NotebookLM

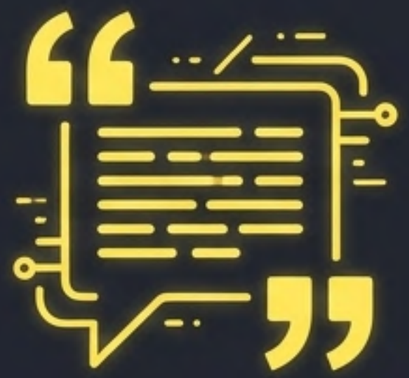


Object

Colecciones de datos mediante pares clave-valor y entidades complejas.

Math

El objeto global para operaciones y utilidades matemáticas avanzadas.



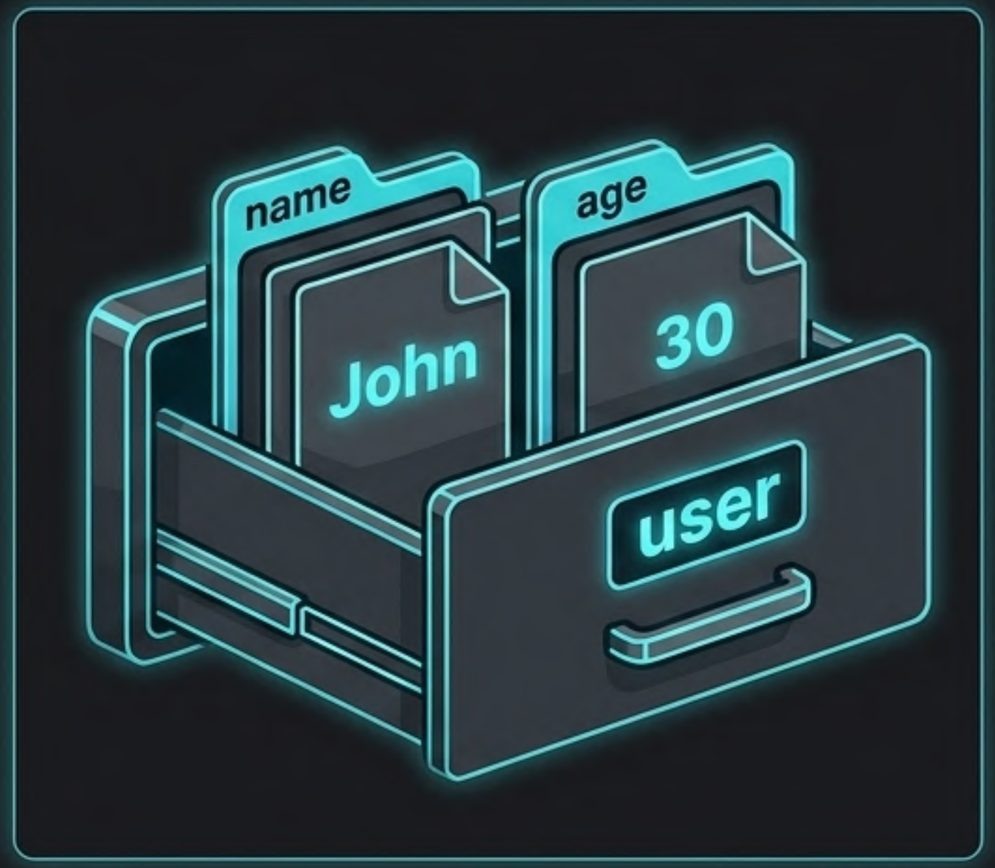
String

Creación, acceso, inmutabilidad y métodos de cadenas de texto.

NotebookLM

¿Qué es un Objeto en JS? (El Modelo Mental)

Un objeto almacena colecciones de datos mediante pares clave: valor. Es la estructura fundamental para organizar datos relacionados.




```
let user = {
    name: 'John',          // clave 'name', valor 'John'
    age: 30,               // clave 'age', valor 30
    'likes birds': true  // claves multi-palabra requieren comillas
};
```

NotebookLM

Accediendo a Propiedades

Notación de Punto (.)

Más limpia y común. Requiere que la clave sea un identificador válido (sin espacios, no empieza con números).

```
console.log(user.name);   // Output: 'John'

delete user.age;          // Elimina la propiedad
```

Notación de Corchetes ([ ])

Permite nombres de propiedades dinámicos o que contienen espacios. Evalúa la expresión dentro de los corchetes.

```
console.log(user['likes birds']);   // Output: true

let key = 'name';
console.log(user[key]);             // Output: 'John'
```

NotebookLM

Sintaxis Avanzada

Propiedades Calculadas (Computed Properties)

Permite usar una variable como nombre de la clave durante la creación del objeto dinámicamente.

```
let fruit = 'apple';
let bag = {
    [fruit]: 5 // La clave se toma de la variable
};
console.log(bag.apple); // Output: 5
```

Atajos de Valor (Property Value Shorthand)

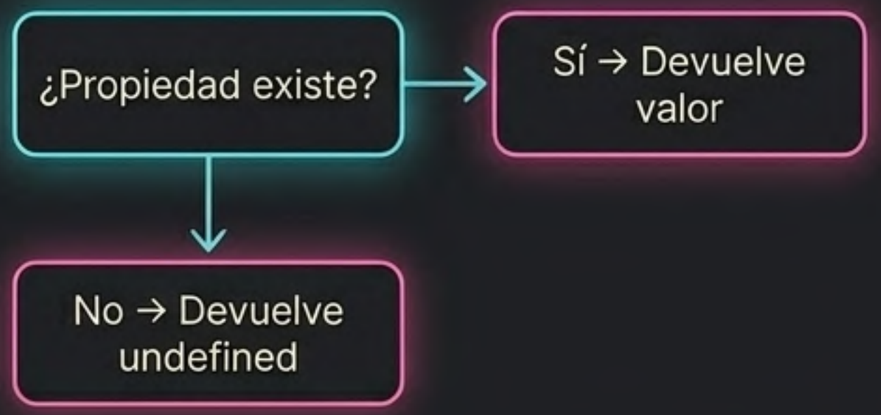
Si la variable y la clave tienen el mismo nombre, podemos omitir la repetición.

```
function makeUser(name, age) {
    return { name, age };
    // En lugar de { name: name, age: age }
}
```

NotebookLM

Comprobación de Existencia y Operador ‘in’

Leer una propiedad que no existe devuelve undefined. El operador 'in' verifica de forma exacta si la clave existe, cubriendo casos límite.



⚠️ ¿Qué pasa si el valor asignado ES **undefined**? → Usa el operador 'in'.

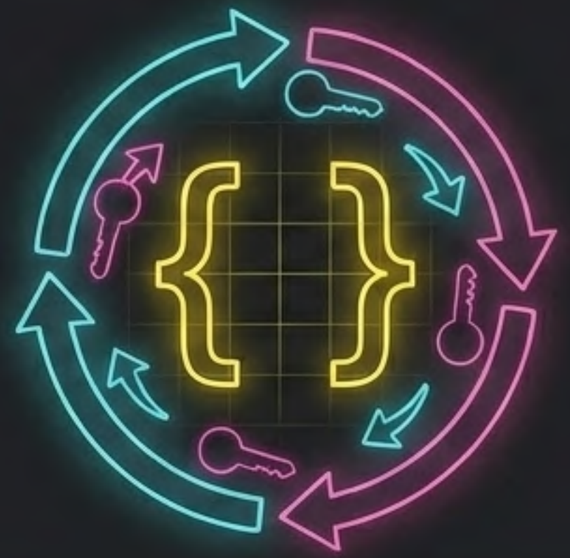
```
let obj = { test: undefined };

// Verificación común (Falla en este caso)
console.log(obj.test === undefined);
// true (¿No existe o es undefined?)

// Operador 'in' (Exacto)
console.log('test' in obj); // Output: true (Sí existe)
console.log('age' in obj);  // Output: false
```

NotebookLM

Iteración con el bucle for..in



⚠️ **¡Atención al orden!** Las claves numéricas enteras (ej. '49') se ordenan **automáticamente de menor a mayor**. Las demás respetan el orden de creación.

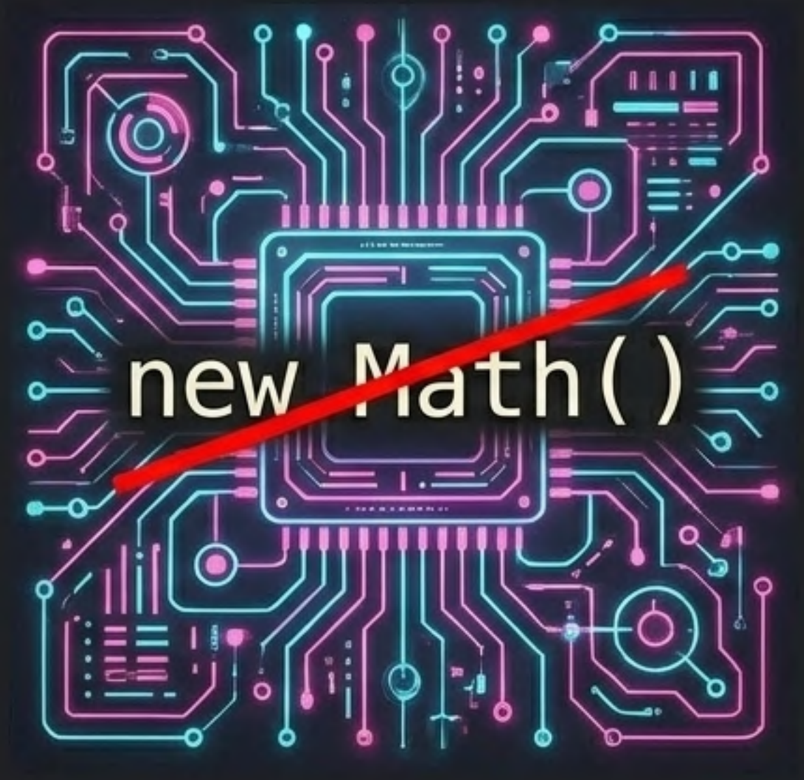
```javascript
let codes = {
    '49': 'Germany',
    '41': 'Switzerland',
    '1': 'USA'
};

for (let key in codes) {
    console.log(key);
    // Output: 1, 41, 49
    // (¡Se ordenaron automáticamente!)
}
```

NotebookLM

Introducción a Math (Objeto Global)

Math es un objeto global integrado. No es una función constructora.
Contiene propiedades y métodos estáticos para operaciones de precisión.




```
// Acceso directo a constantes, sin instanciar
console.log(Math.PI);
// Output: 3.141592653589793

console.log(Math.E);
// Output: 2.718281828459045

// Comprobación del tipo de dato
console.log(typeof Math);
// Output: 'object'
```

NotebookLM

Métodos de Redondeo

Comparación del comportamiento de métodos clave usando los números flotantes 3.14 y -3.14.

	Math.round()	Math.ceil()	Math.floor()	Math.trunc()
	Al entero más cercano.	Hacia arriba (techo).	Hacia abajo (piso).	Elimina los decimales.
Input 3.14	-> 3	-> 4	-> 3	-> 3
Input -3.14	-> -3	-> -3	-> -4	-> -3

```
// Ojo con los negativos: floor vs trunc
console.log(Math.floor(-3.14)); // Output: -4
console.log(Math.trunc(-3.14)); // Output: -3
```

NotebookLM

Potencias, Raíces y Extremos

Potencias

```
Math.pow(base, exponente)
Math.pow(2, 3); // Output: 8
```

Raíces

```
Math.sqrt(16); // Cuadrada -> 4
Math.cbrt(27); // Cúbica -> 3
```

Máximo

```
Math.max(a, b, c...)
Math.max(5, 10, -3); // Output: 10
```

Mínimo

```
Math.min(a, b, c...)
Math.min(5, 10, -3); // Output: -3
```

NotebookLM

Generación Aleatoria

Math.random() genera un decimal pseudoaleatorio entre 0 (incluido) y 1 (excluido).
Para rangos útiles, se combina con Math.floor().

Transformación con Math.floor y la fórmula

```
// Número base (ej. 0.843...)
console.log(Math.random());

// FÓRMULA MAESTRA:
// Entero aleatorio entre 'min' y 'max'
let min = 10;
let max = 50;

let randomNum = Math.floor(Math.random() *
    (max - min)) + min;

console.log(randomNum);
// Output: Número entero entre 10 y 49
```

NotebookLM

Creación e Inmutabilidad de Strings

Creación

- Comillas Simples (' ')
- Comillas Dobles (" ")
- Template Literals (` `) + variables con ${}

Inmutabilidad

Inmutabilidad: Los caracteres no se pueden cambiar por índice. Se debe reasignar la variable entera.

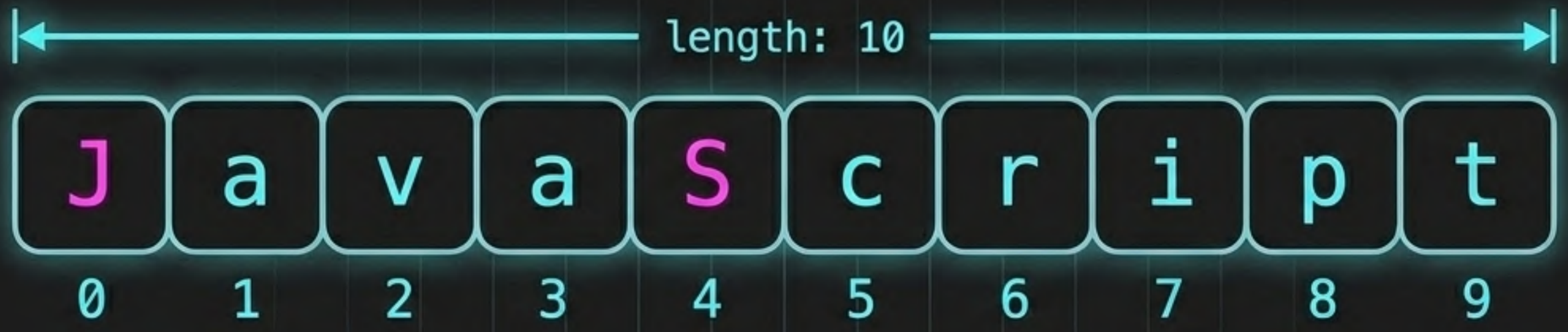
```
let message = 'Hello';
message[0] = 'J';
// Falla silenciosamente (o error en Strict Mode)

console.log(message); // Output: 'Hello'

// Solución correcta: Reasignar
message = 'Jello';
```

NotebookLM

Acceso y Longitud (Tratando Strings como Arrays)




```javascript
let str = 'JavaScript';

// Acceso por índice
console.log(str[0]);              // Output: 'J'
console.log(str.charAt(4));       // Output: 'S'

// Último carácter dinámico
console.log(str[str.length - 1]); // Output: 't'
```

NotebookLM

Matriz de Métodos de String (Cheat Sheet)

Búsqueda (Search)	
`str.includes('a')`	Retorna booleano si encuentra la subcadena.
`str.search('a')`	Retorna la posición (índice) de la primera coincidencia.

Extracción (Extraction)	
`str.substring(start, end)`	Extrae parte del string.
`str.slice(start, end)`	Igual, pero acepta índices negativos (desde el final).
`str.split(' ')`	Convierte el string en un Array, cortando por el separador.

Transformación (Transformation)	
`str.toUpperCase() / toLowerCase()`	Cambia mayúsculas/minúsculas.
`str.trim()`	Remueve espacios en blanco de los extremos.
`str.replace(old, new)`	Reemplaza una subcadena por otra.

NotebookLM

Cadenas Multilínea y Caracteres de Escape

El carácter de escape '\\' permite insertar caracteres especiales. Los Template Literals solucionan el formato multilínea de forma elegante.

Manejo de Comillas y Saltos

```
1  // Comillas anidadas (Caracter de Escape)
2  let texto = 'Él dijo: \"JavaScript es
3  genial\"';
4  // Multilínea tradicional
5  let multiline = 'Línea 1\n
6                   Línea 2\n
7                   Línea 3';
```

Template Literals

```
1  // Multilínea moderna (Backticks)
2  let moderna = `Línea 1
3                 Línea 2
4                 Línea 3`;
5  // El salto de línea físico se conserva
```

NotebookLM